

```

• // `this` points to the vm instance
•
•
• return this.message.split('').reverse().join('')
• }
• }
• })

```

Result:

Original message: "Hello"

Computed Reverse Message: "olleH"

Here, computed property has been declared with "reversed message". The function is used as a getter function for the property.

```

• console.log(vm.reversedMessage) // => 'olleH'
• vm.message = 'Goodbye'
• console.log(vm.reversedMessage) // => 'eybdooG'

```

Here, you can open the console and conduct your own example with **vm**. The value of **vm.reversed** message is dependent on the value of **vm.message**.

Just like a normal property, you can data bind with computed property in the given template. In this, **vm.reversedMessage** depends on **vm.message**, so it would automatically update the binding that depends on **vm.reversedMessage** whenever **vm.message** changes.

Here, the best part is that a declarative relationship has been created with this dependency. Also, the computed getter function has no side effects. In fact, it is much easier to understand and test simultaneously.

Computed Property vs. Method

It should be noted that here we can stimulate the same result with the following codes:

```

• <p>Reversed message: "{{ reverseMessage() }}"</p>
• // in component
• methods: {
•   reverseMessage: function () {
•     return this.message.split('').reverse().join('')
•   }
• }

```

In the same way, we can define a method. Both would approach the same results. However, again the difference is that the computed properties are cached and have reactive dependencies. A computed property would only re-evaluate when any of its reactive dependencies change. This means